

HTML & CSS - 2 Hour Lecture Plan

HTML & CSS — 2-Hour Lecture Plan

Full teaching notes — explanations, analogies, live-demo ideas & code, ready to teach straight from

Lecture Overview

Time	Topic
0:00 - 0:10	Quick Recap: HTML vs CSS, 3 ways to link CSS
0:10 - 0:25	Semantic Tags
0:25 - 0:40	CSS Selectors & Specificity
0:40 - 1:00	CSS Box Model
1:00 - 1:10	Break
1:10 - 1:30	Flexbox Basics
1:30 - 1:40	Positioning (relative, absolute, fixed)
1:40 - 1:50	Responsive Basics: units & media queries
1:50 - 2:15	Hands-on Practice: Profile Card + Mini Layout
2:15 - 2:20	Wrap-Up & Q&A

(Note: this totals slightly over 2 hours with buffer built in — trim the “Positioning” section first if you’re short on time, since Flexbox alone covers most alignment needs.)

1. Quick Recap (10 min)

Ask the class: “**What’s the difference between HTML and CSS?**”
Someone will say HTML = structure, CSS = style. Reinforce with an analogy:

| “HTML is the skeleton, CSS is the skin/clothes.”

Also ask how CSS is linked to HTML — there are 3 ways:

- **Inline** — `style="color:red;"` directly on a tag (quick, but messy — avoid in real projects)
- **Internal** — a `<style>` tag inside `<head>`
- **External** — a separate `.css` file linked via `<link rel="stylesheet" href="style.css">` — **best practice**, mention this is what real projects use

```
<!-- Inline -->
<p style="color:red;">This is red text</p>

<!-- Internal -->
<head>
  <style>
    p { color: red; }
  </style>
</head>
```

```
<!-- External (preferred) -->
<head>
  <link rel="stylesheet" href="style.css">
</head>
```

Teaching tip: Say clearly — “You’ll almost always use external CSS in real projects. Inline is for quick testing only.”

2. Semantic Tags (15 min)

Explain: before semantic tags, people used `<div>` for everything, which told the browser nothing about meaning. Semantic tags tell you *and* the browser what a chunk of content is.

Walk through each one out loud:

- `<header>` — top section, usually logo/title
- `<nav>` — navigation links
- `<main>` — the primary content of the page (only one per page)
- `<section>` — a thematic grouping of content
- `<article>` — self-contained content (a blog post, a news story)
- `<aside>` — side content, like a sidebar or related links
- `<footer>` — bottom section, copyright/contact info

“You could technically build the whole page with `<div>`s, but semantic tags make your code easier to read, better for accessibility tools like screen readers, and better for SEO.”

```
<header><h1>My Website</h1></header>
<nav><a href="#">Home</a> <a href="#">About</a></nav>
<main>
  <section>
    <article>Blog post content here</article>
  </section>
  <aside>Related links</aside>
</main>
<footer>&copy; 2026</footer>
```

Live-code this example, and show them it looks the same as using `<div>`s visually — the difference is invisible but important. Open DevTools and show the “Accessibility” or “Elements” tree to prove the browser actually understands the structure differently.

3. CSS Selectors & Specificity (15 min)

(New section — students need this before Box Model/Flexbox make full sense, since every property they write needs a selector to target.)

Say: “Before we style anything, you need to know how to *target* the right element. This is what selectors do.”

```
/* Element selector - targets ALL <p> tags */
p { color: blue; }

/* Class selector - targets elements with class="highlight" */
.highlight { background-color: yellow; }

/* ID selector - targets the ONE element with id="main-title" */
#main-title { font-size: 32px; }

/* Descendant selector - targets <a> tags INSIDE <nav> */
nav a { text-decoration: none; }

/* Multiple classes */
.btn.primary { background-color: blue; color: white; }
```

Specificity — which rule wins?

Say plainly: “If two rules target the same element, the more *specific* one wins.”

Rough order (lowest to highest priority): 1. Element selectors (p, div)
2. Class selectors (.highlight) 3. ID selectors (#main-title) 4. Inline styles (always win, avoid relying on this)

Live demo: Write `p { color: red; }` and `.highlight { color: blue; }`, apply both to the same `<p class="highlight">`, show that blue wins because class beats element.

Teaching tip: Don’t go too deep into specificity math (like the 0,0,1,0 point system) — just the general rule “class beats element, ID beats class” is enough for now.

4. CSS Box Model (20 min)

This is **the most important mental model in CSS**. Draw four nested boxes on the whiteboard (or open DevTools):

margin -> border -> padding -> content

Explain each from inside out:

- **Content** — the actual text/image
- **Padding** — space inside the border, between border and content
- **Border** — the visible line around the element
- **Margin** — space outside the border, pushing other elements away

“Padding is like a cushion inside a box. Margin is the space you leave between two boxes on a shelf.”

```
.box {  
  width: 200px;  
  padding: 20px;  
  border: 2px solid black;  
  margin: 15px;  
}
```

Then open DevTools on any live webpage (F12 → Inspect element) and hover over an element — the browser highlights padding/border/margin in different colors. **This visual really cements it — don’t skip this step.**

box-sizing — the classic gotcha

Say: “By default, width only applies to content — padding and border get *added* on top, which confuses everyone at first.”

```
/* Default behavior - width becomes 200 + 40(padding) + 4(border) =  
244px total */  
.box {  
  width: 200px;  
  padding: 20px;  
  border: 2px solid black;  
}  
  
/* Fix - width INCLUDES padding and border, stays exactly 200px */  
.box {  
  box-sizing: border-box;  
  width: 200px;  
  padding: 20px;  
  border: 2px solid black;  
}
```

Teaching tip: Mention that most developers add `* { box-sizing: border-box; }` at the top of every project to avoid this confusion entirely — show it live, resize the box, and watch the total size stop changing.

Break (10 min)

5. Flexbox Basics (20 min)

“Before flexbox, centering something vertically was a nightmare. Flexbox fixed that.”

Teach it as: flex turns a container into a row or column that automatically arranges its children.

Go through properties one by one, **changing one value at a time live** so students see the effect:

- `display: flex` → children now line up in a row automatically
- `flex-direction: row | column` → controls the direction
- `justify-content` → aligns items horizontally (main axis) — center, space-between, space-around
- `align-items` → aligns items vertically (cross axis) — center, flex-start, flex-end
- `gap` → spacing between items, no need for margins anymore
- `flex-wrap: wrap` → lets items drop to the next line instead of overflowing (mention briefly)

```
.container {  
  display: flex;  
  justify-content: center;  
  align-items: center;  
  gap: 10px;  
}
```

Live demo idea: Put 3 `<div>` boxes with background colors inside a flex container, then toggle `justify-content` and `align-items` values live so students visually see the boxes move.

```
<div class="container">  
  <div class="box" style="background:tomato;">1</div>  
  <div class="box" style="background:gold;">2</div>  
  <div class="box" style="background:skyblue;">3</div>  
</div>
```

Teaching tip: This is the “wow” moment of the whole lecture — spend real time here, let students guess what each property does before you show them.

6. Positioning: relative, absolute, fixed (10 min)

(New section — commonly needed alongside Flexbox, e.g. for badges, tooltips, sticky headers.)

Say: “Flexbox handles most layouts, but sometimes you need to place something in an *exact* spot — that’s what position is for.”

```
/* relative - moves the element relative to its NORMAL position,  
leaves a gap behind */  
.box-relative {  
  position: relative;
```

```

    top: 10px;
    left: 20px;
  }

  /* absolute - removes from normal flow, positions relative to
  nearest positioned ancestor */
  .badge {
    position: absolute;
    top: 0;
    right: 0;
  }

  /* fixed - stays in place even when scrolling (e.g. sticky navbars)
  */
  .navbar {
    position: fixed;
    top: 0;
    width: 100%;
  }

```

Teaching tip: Use a concrete example — a notification badge (small red circle) positioned on the corner of a profile picture using `position: absolute` on the badge and `position: relative` on the picture's container. This is a real pattern they'll use constantly.

7. Responsive Basics: Units & Media Queries (10 min)

(New section — without this, students can't make anything that works on mobile, which is expected of any modern site.)

Relative units (briefly)

- px — fixed size, doesn't scale
- % — relative to the parent element
- rem — relative to the root font size (most recommended for consistent scaling)

```

body { font-size: 16px; }
h1 { font-size: 2rem; } /* = 32px, scales if root font size changes
*/

```

Media queries — making layouts respond to screen size

```

.container {
  display: flex;
  flex-direction: row;
}

/* On screens smaller than 600px, stack items vertically instead */
@media (max-width: 600px) {
  .container {
    flex-direction: column;
  }
}

```

Live demo: Resize the browser window (or use DevTools' device toolbar) with a flex row container, show it staying a row until it hits the breakpoint and switches to a column. This is the moment students realize how "responsive" sites actually work.

Teaching tip: Keep this section light — the goal is just "I know this exists and roughly how it works," not mastery. It gives a natural bridge to a future class on responsive design.

8. Hands-on Practice (25 min)

Let students build a **profile card + mini layout page**, combining everything from today:

“Build a personal profile page with: 1. A <header> with your name as <h1>, and a <nav> with 3 fake links 2. A profile card inside <main> — image + name + short bio, centered on the page using flexbox, with padding/border/margin applied 3. A small ‘badge’ (e.g. a colored circle with text like ‘Available’) positioned on the corner of the profile image using position: absolute 4. A <footer> with a copyright line 5. Bonus if time allows: add a media query so the card becomes full-width on small screens”

Walk around / monitor, help as needed. This is where the box model, flexbox, and positioning concepts all get cemented through doing, not listening.

Sample starter structure (don’t give this away until after they attempt it):

```
<header><h1>Jane Doe</h1><nav><a href="#">Home</a> <a href="#">About</a> <a href="#">Contact</a></nav></header>
<main>
  <div class="profile-card">
    <div class="image-wrapper">
      
      <span class="badge">Available</span>
    </div>
    <h2>Jane Doe</h2>
    <p>Frontend developer who loves clean design.</p>
  </div>
</main>
<footer>&copy; 2026 Jane Doe</footer>

* { box-sizing: border-box; }
.profile-card {
  display: flex;
  flex-direction: column;
  align-items: center;
  width: 300px;
  margin: 40px auto;
  padding: 20px;
  border: 1px solid #ccc;
  border-radius: 10px;
}
.image-wrapper { position: relative; }
.badge {
  position: absolute;
  top: 0;
  right: 0;
  background: green;
  color: white;
  padding: 4px 8px;
  border-radius: 12px;
  font-size: 12px;
}
@media (max-width: 400px) {
  .profile-card { width: 90%; }
}
```

9. Wrap-Up + Q&A (5 min)

Quickly recap the big ideas of the day:

- **Semantic tags** = meaningful structure
- **Selectors & specificity** = how to target and prioritize styles
- **Box model** = spacing logic (content → padding → border → margin)
- **Flexbox** = easy alignment
- **Positioning** = exact placement when needed
- **Responsive basics** = making layouts adapt to screen size

Homework: Extend today's profile page — add a second card (e.g. "skills" or "projects" section) using flexbox to lay out multiple cards side by side, and make sure it looks good on both desktop and mobile width using a media query.

Tips for Teaching This Session

- Live-code everything — don't paste finished HTML/CSS, type it out so students see the process, including small mistakes (e.g. forgetting a closing brace) — this is realistic and builds debugging intuition.
- DevTools (F12) is your best teaching tool today — use it constantly to show the box model highlighting and to live-tweak CSS values without reloading the page.
- The Flexbox section is the highlight of the lecture — don't rush it. Let students predict what a property will do *before* you change the value.
- If short on time, cut the "Positioning" section first (it's useful but least essential), then trim the media query demo to just one resize example.
- Keep reinforcing the box model diagram (margin -> border -> padding -> content) throughout the lecture — it's the concept most likely to be forgotten, so repetition helps.